

Code-Dokumentation und Annahmen Case Study Innovationspark Jüchen

Inhalt

1	How to Run.....	3
1.1	Run Web GUI.....	3
1.2	Run Sensitivity Analysis	3
2	Code-Dokumentation.....	4
2.1	Programmablauf und Systemarchitektur	4
2.2	PV Modulator.....	5
2.3	Energielandschaft.....	10
2.4	Industriegebiet.....	12
2.5	Green Energy Hub	14
2.6	Batterie.....	15
2.7	Input/Output Types.....	17
2.8	Graphical User Interface	19
2.8.1	Unterstützende Webfunktionen	21
2.9	Simulation	24
2.10	Sensitivitätsanalyse.....	28
2.11	Hilfsfunktionen.....	29
2.11.1	Geometrie-Utilities.....	29
2.11.2	Wetterdatenmodul.....	30
3	Annahmen Case Study Innovationspark Jüchen.....	32
3.1	Solarautobahn.....	32
3.2	Energielandschaft.....	33
3.3	Green Energy Hub	35
3.4	Industriegebiet.....	38
3.4.1	Auswahl Lastgänge und Erläuterung Skalierungsfaktor	39

3.4.2	Entsorgung.....	41
3.4.3	Stahl 1	41
3.4.4	Transportlogistik und Lagerung.....	42
3.4.5	Schrott- und Metallhandel	42
3.4.6	Lagerung	43
3.4.7	Büro	43
3.4.8	Logistik Lebensmittel.....	44
3.4.9	Stahl 2	44
3.4.10	Weitere Lastgänge	45
3.5	Stadtteilentwicklung Jüchen Süd.....	46

1 How to Run

Mehr Informationen in den jeweiligen Kapiteln.

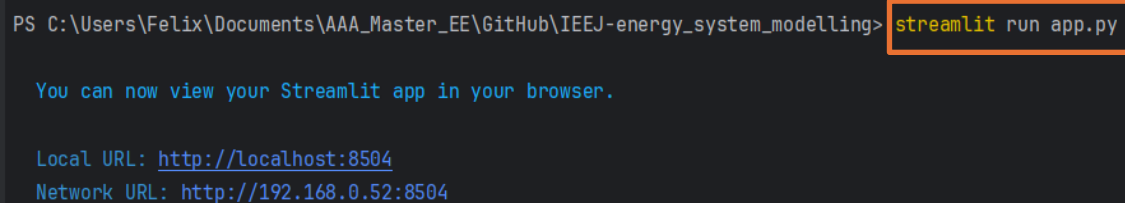
Grundlegende Anwendung funktioniert wie folgt:

1.1 Run Web GUI

Über das Terminal im Projektordner folgenden Befehl ausführen.

bash:

```
streamlit run Web_Gui.py
```



```
PS C:\Users\Felix\Documents\AAA_Master_EE\GitHub\IEEJ-energy_system_modelling> streamlit run app.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8504
Network URL: http://192.168.0.52:8504
```

Alternativ kann das Programm über die im Projekt enthaltene Web_Gui.bat gestartet werden.

1.2 Run Sensitivity Analysis

Sensitivitätsanalyse wird durch das Skript core/auxiliary/**sensitivity_analysis.py** ausgeführt. Die gewünschten Szenarios sind im Skript zu wählen.

2 Code-Dokumentation

2.1 Programmablauf und Systemarchitektur

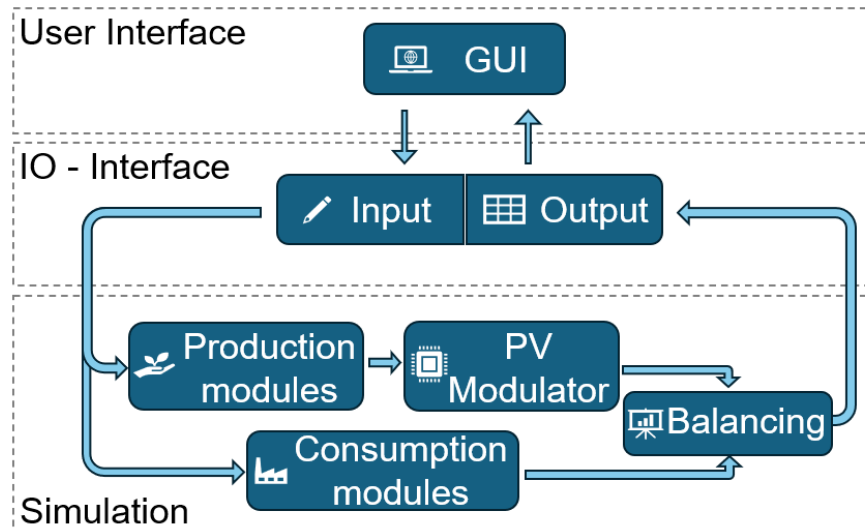
Dokumentation der klassenbasierten Module

- Programmiersprache: Python
- Verfügbar auf GitHub von Moritz End (moend95)
 - IEEJ-energy_system_modelling
 - https://github.com/moend95/IEEJ-energy_system_modelling

Für die Verwendung des Codes in Python wurden folgende Bibliotheken und Software Versionen verwenden:

Paket	Version
Python	3.10
folium	0.20.0
geopandas	0.13.2
matplotlib	3.10.3
numpy	1.26.4
osmnx	2.0.5
pandas	2.1.4
pillow	11.2.1
plotly	6.1.1
pvinfos	1.5.2
pvlb	0.12.0
pyproj	3.7.1
scipy	1.13.1
Shapely	1.8.5.post1
streamlit	1.55.0
streamlit-folium	0.26.2
wetterdienst	0.89.0

Die Abbildung veranschaulicht den grundsätzlichen Ablauf des Programms. Das Framework ist in drei Ebenen gegliedert: die grafische Benutzeroberfläche, die Input-/Output-Schnittstelle und den Simulationskern. In der GUI werden die Szenarioparameter durch den Nutzer festgelegt und in einer strukturierten Input-Datenstruktur gebündelt. Diese Eingaben werden anschließend an die Simulationsmodule übergeben, in denen Erzeugung, Verbrauch und optional Speicher separat berechnet und in einem Bilanzierungsschritt zusammengeführt werden.



Die resultierenden Kennwerte und Zeitreihen werden in einer Output-Datenstruktur gespeichert und an die GUI zurückgegeben, wo sie visualisiert werden. Durch diese Trennung von Eingabe, Berechnung und Ausgabe bleibt das Framework modular, übersichtlich und leicht erweiterbar.

2.2 PV Modulator

module_pv_modulator.py

Das PV-Modul stellt eine klassenbasierte Implementierung zur Simulation von Photovoltaikanlagen entlang linearer Geometrien dar. Es umfasst die Generierung von Anlagenpositionen, die Bestimmung der Modulorientierung sowie die zeitaufgelöste Leistungsberechnung auf Basis von Wetterdaten.

Der Aufbau des Moduls gliedert sich in mehrere zentralen Klassen:

- **PVType** zur Konfiguration unterschiedlicher Anlagentypen
- **AzimuthCalculator** zur geometrischen Vorverarbeitung und Bestimmung der Modulausrichtung
- **AzimuthVisualizer** zur optionalen grafischen Kontrolle sowie
- **PV_System** als übergeordnete Steuerklasse für Punktgenerierung und Ertragssimulation

Die detaillierte Beschreibung der einzelnen Klassen erfolgt in den nachfolgenden Abschnitten.

Class PVType (Konfigurationsebene)

Die *Dataclass PVType* beschreibt einen vollständigen PV-Anlagentyp. Sie dient als strukturierte Sammlung aller technischen und geometrischen Parameter und ermöglicht die konsistente Definition unterschiedlicher Systemvarianten.

Wesentliche Parameter sind:

- azimuth, tilt: Orientierung der Module
- module, inverter: Auswahl aus pvlib-Datenbanken
- modules_per_string, strings_per_inverter: elektrische Verschaltung
- row_height, row_width: geometrische Dimensionen
- bifaciality, gcr, albedo: Parameter für bifaziale Systeme
- mount_type: „fixed“ oder „singleaxis“

Zur Vereinfachung werden standardisierte PV-Module, Wechselrichter und Temperaturmodelle verwendet. Für eine präzisere Simulation sollten diese jedoch an den jeweiligen Anwendungsfall angepasst werden.

module	Canadian_Solar_CS6X_300M__2013_
inverter	ABB__PVI_3_0_OUTD_S_US__208V_
temp_model	open_rack_glass_glass

Die Liste PV_TYPEN enthält vordefinierte Konfigurationen für die im Projekt betrachteten Anwendungen.

ID	Name	Tilt [°]	Azimut	Mount-Type	Bifazial
1	Windschutzwand	90	variabel	fixed	True
2	Schallschutzwand	90	variabel	fixed	None
3	Schallschutzwand_Dach	30	variabel	fixed	None
4	Böschung	30	variabel	fixed	None
5	Freifläche /Agri Tracking	variabel	variabel	singleaxis	True
6	Agri_vertikal	90	90	Fixed	None
7	Haus_Dach	30	180	Fixed	None

Class *AzimuthCalculator* (Geometrieverarbeitung)

Die Klasse *AzimuthCalculator* übernimmt die Verarbeitung der Geometriedaten und die Ableitung der Modulorientierung. Sie bereitet lineare *GeoJSON*-Geometrien für die PV-Simulation auf, indem entlang der Geometrie diskrete Simulationspunkte erzeugt und mit einem Azimutwert versehen werden. Die Ergebnisse werden als *GeoDataFrame* sowie als CSV-Datei bereitgestellt und bilden die Schnittstelle zur energetischen Simulation im *PV_System*.

Die Initialisierung erfolgt über den Pfad zur *GeoJSON*-Datei, die Schrittweite der Punktgenerierung, den gewählten PV-Typ sowie einen Ausgabepfad.

Die Verarbeitung umfasst vier zentrale Schritte: Einlesen und Vereinheitlichen der Geometrie, Erzeugung diskreter Punkte, Bestimmung des Azimuts sowie Export der Ergebnisse.

Die Methode *load_geometry()* liest die Geometrie ein und transformiert sie in ein metrisches Koordinatensystem.

Mit *interpolate_points()* werden entlang der Geometrie Punkte im Abstand der definierten Schrittweite erzeugt. Diese bestimmen die räumliche Auflösung der späteren Simulation.

Die Methode *calculate_azimuth()* weist jedem Punkt eine Modulorientierung zu. Wenn im *PVType* ein fester Azimutwert hinterlegt ist, wird dieser unverändert übernommen. Wenn dagegen *pv_type.azimut = "variabel"* gesetzt ist, wird der Azimut aus der lokalen Geometrie abgeleitet.

Die berechneten Punkte und Azimutwerte werden anschließend in ein *GeoDataFrame* überführt und als CSV-Datei exportiert. Dieses Format dient als Eingangsgröße für die nachfolgende PV-Simulation im *PV_System*.

Class *AzimuthVisualizer* (optionale Visualisierung)

Die Klasse *AzimuthVisualizer* dient der grafischen Darstellung der durch den *AzimuthCalculator* erzeugten PV-Punkte und deren Orientierung. Sie ist nicht Teil der eigentlichen Simulation, sondern ein optionales Werkzeug zur visuellen Kontrolle der Geometrieverarbeitung und *Azimutberechnung*. Die Klasse erwartet eine Instanz von *AzimuthCalculator*, aus der die berechneten Punkte (*point_azimut_list*) sowie die ursprüngliche Geometrie (*gdf*) übernommen werden.

Die Methode *plot()* erzeugt eine zweidimensionale Darstellung mit folgenden Elementen:

- Geometrie: Darstellung der ursprünglichen Linie oder Fläche (grau)
- PV-Punkte: diskretisierte Positionen der Anlagen (rot)
- Azimutrichtung: Pfeile zur Visualisierung der Modulorientierung (blau)

Die Richtung der Pfeile wird aus dem Azimutwinkel berechnet. Dazu erfolgt eine Umrechnung von Grad in kartesische Komponenten. Zur Reduktion der visuellen Überlagerung kann über den Parameter *every_nth_arrow* gesteuert werden, dass nur jeder n-te Pfeil dargestellt wird.

Die Klasse dient ausschließlich der Validierung und Visualisierung der berechneten Geometrie und hat keinen Einfluss auf die energetische Simulation.

Class *PV_System*

Die Klasse *PV_System* stellt die zentrale Steuerkomponente des PV-Moduls dar. Sie verbindet die geometrische Vorverarbeitung mit der energetischen Simulation und bildet damit die Hauptschnittstelle des Systems. Ein *PV_System* wird durch die zugrunde liegende Geometrie, eine Liste von PV-Typen, die Schrittweite, Wetterdaten sowie ein Ausgabeverzeichnis definiert.

Die Klasse übernimmt zwei Hauptaufgaben: die Generierung von Simulationspunkten für jeden PV-Typ sowie die anschließende Berechnung und Aggregation der elektrischen Leistung zu zeitaufgelösten Leistungsreihen und Jahresenergien.

Die Methode *generate_pv_points()* erzeugt für jeden PV-Typ diskrete Simulationspunkte entlang der Geometrie, indem der *AzimuthCalculator* verwendet wird. Die Ergebnisse werden als *GeoDataFrames* zurückgegeben und zusätzlich als CSV-Dateien gespeichert.

Zur Abbildung der Anlagenkonfiguration wird mit *_build_mount()* das entsprechende Montagesystem auf Basis der PV-Typ-Parameter erzeugt. Dabei wird zwischen fest installierten (*FixedMount*) montierten Anlagen und nachgeführten Systemen (*SingleAxisTrackerMount*) unterschieden. Für Tracking-Systeme werden die Parameter *axis_tilt*, *axis_azimuth*, *max_angle*, *backtrack* und gegebenenfalls *gcr* aus dem *PVType* übernommen. Für fest montierte Systeme werden die Werte *tilt* und *azimuth* aus der Punktdatensatz verwendet.

Die Methode `_simulate_monofacial_point()` berechnet die elektrische Leistung eines einzelnen Punkts eines monofazialen Systems unter Verwendung der *pvlib-ModelChain* auf Basis der Standortkoordinaten und Wetterdaten. Sie stellt die Standardberechnung für alle nicht detailliert bifazial modellierten Systeme dar.

Die Methode `_simulate_bifacial_point()` erweitert die Simulation um eine explizite Berücksichtigung der Rückseitenbestrahlung mittels *pvfactors*. Dabei werden geometrische Effekte, Albedo, Ground Coverage Ratio (GCR) sowie der Sonnenstand berücksichtigt. Für jeden Zeitschritt wird eine effektive Einstrahlung bestimmt, die sowohl Vorder- als auch Rückseitenanteile enthält und zur Leistungsberechnung verwendet wird. Dieser Ansatz ermöglicht eine realistischere Abbildung bifazialer Systeme, ist jedoch mit erhöhtem Rechenaufwand verbunden und wird aktiviert, wenn `is_bifacial_detailed = True` gesetzt ist.

Die Methode `simulate_pv_type_energy()` liest die erzeugten Simulationspunkte eines PV-Typs ein, berechnet die Leistung für jeden Punkt und aggregiert die Ergebnisse zu einer Gesamtleistungszeitreihe. Die Skalierung erfolgt entsprechend der elektrischen Verschaltung (Module pro String und Strings pro Wechselrichter). Die jährliche Energie wird durch Integration der Leistungszeitreihe bestimmt.

Für bifaziale Systeme stehen somit zwei Modellierungsansätze zur Verfügung:

- **Vereinfachter Ansatz:** Die Simulation erfolgt zunächst monofazial, anschließend wird ein konstanter Mehrertragsfaktor auf die Gesamtenergie angewendet. die zeitaufgelöste Leistungszeitreihe bleibt dabei unverändert.
- **Detaillierter Ansatz:** Die Rückseitenbestrahlung wird mit *pvfactors* unter Berücksichtigung von Geometrie, Albedo, GCR und Sonnenstand zeitaufgelöst berechnet und in eine effektive Einstrahlung überführt.

Die Methode liefert sowohl die jährliche Energie (`energy_type_kWh`) als auch die aggregierte AC-Leistungszeitreihe (`total_type_ac_w`) und ermöglicht damit eine konsistente Bewertung unterschiedlicher PV-Systemkonfigurationen.

2.3 Energielandschaft

module_energy_landscape.py

Das Modul `module_energy_landscape.py` implementiert eine modulare Verarbeitungspipeline zur parametrisierten Erzeugung von Photovoltaik-Flächen auf landwirtschaftlichen Flächen. Es dient als eigenständiges Werkzeug zur Einlesung von *GeoJSON-Flächen*, zur Konfiguration von PV-Modultypen, zur Berücksichtigung landwirtschaftlicher Maschinenparameter sowie zur Erzeugung von PV-Modulgittern und PV-Reihen als *GeoJSON-FeatureCollections*. Damit ermöglicht das Modul eine konsistente und reproduzierbare Generierung von PV-Layouts ohne manuelle Koordinateneingaben.

Zu Beginn wird eine eingebettete GeoJSON-Fläche (`EMBEDDED_GEOJSON`) geladen, die das zu beplanende Polygon in WGS84-Koordinaten beschreibt. Die Klasse *GeoJSONReader* übernimmt das Parsen der *GeoJSON*-Daten und reprojiziert die Fläche automatisch in ein metrisches UTM-Koordinatensystem. Hierbei wird die passende UTM-Zone basierend auf dem *Polygon-Zentroid* gewählt, wodurch präzise Flächen- und Längenberechnungen möglich werden. Der Transformationsmechanismus liefert neben dem *Shapely-Polygon* in UTM auch Transformer-Objekte für die Rückprojektion nach WGS84.

Die Konfiguration des PV-Systems erfolgt über den *PVTypeSelector*, der den Typ der PV-Module abfragt (horizontal, vertikal oder horizontal mit Single-Axis-Tracking), sowie über den *AusbauStatusSelector*, der den aktuellen Ausbaustatus der Energielandschaft (0 %, 50 %, 100 %) bestimmt. Diese Auswahl wird genutzt, um die erzeugten PV-Reihen gemäß Ausbaustatus zu skalieren, sodass Teilflächen oder vollständige Flächen modelliert werden können.

```
@staticmethod 2 usages 8 jbrinkma
def select(default: str = "horizontal", interactive: bool = True) -> str:
    if not interactive:
        return default

    options = {
        "1": "horizontal",
        "2": "vertical",
        "3": "horizontal_tracking",
    }
    default_key = next((k for k, v in options.items() if v == default), "1")
    prompt = (
        f"Bitte PV-Typ auswählen [Default: {default_key}]\n"
        "1) Horizontale PV-Anlage (aufgeständert, Ost-West)\n"
        "2) Vertikale PV-Anlage (Zaun, Nord-Süd)\n"
        "3) Horizontale PV-Anlage mit Single-Axis-Tracking (aufgeständert, Nord-Süd)\n"
        "Auswahl (1, 2 oder 3): "
    )
    )
```

Zur Berücksichtigung landwirtschaftlicher Rahmenbedingungen werden in der Klasse *TractorParameters* die Traktorbreite, -länge und der Wenderadius abgefragt. Auf Basis dieser Parameter werden Wende- und Fahrspurbreiten inklusive eines Sicherheitszuschlags berechnet. Diese Margen werden anschließend durch den *AreaCalculator* bzw. die Funktion *compute_restflaeche* von der nutzbaren Fläche abgezogen, sodass nur die tatsächlich verfügbare Fläche für die PV-Planung übrig bleibt.

Die Erzeugung der PV-Punkte erfolgt in zwei Stufen: Mit der Klasse *PVGridGenerator* können rasterförmige Punktmuster innerhalb der nutzbaren Fläche erstellt werden, die als *GeoJSON FeatureCollection* exportiert werden. Alternativ generieren die Funktionen *generate_pv_points* und *generate_pv_reihen* die Modulmitten bzw. PV-Reihen entlang paralleler Linien unter Berücksichtigung der gewählten Ausrichtung, des Reihenabstands und des Modulabstands. Dabei werden alle geometrischen Berechnungen in UTM durchgeführt und optional in WGS84 zurückprojiziert. Für jede PV-Reihe werden dabei Schnittpunkte mit dem Polygon berechnet und äquidistante Punkte entlang der Linien erzeugt, sodass ein vollständiges und genau positioniertes PV-Gitter entsteht.

Über die Funktion *run_standalone* wird ein kurzer Ablauf gebündelt, der die eingebettete Fläche einliest, PV-Typ und Traktorparameter abfragt, Wende- und Fahrspurmargen berechnet, die nutzbare Innenfläche bestimmt und darauf die PV-Punkte bzw. PV-Reihen generiert. Damit stellt das Modul eine durchgängige, reproduzierbare Pipeline für die parametrisierte Planung von Agri-PV-Flächen bereit und ermöglicht die flexible Integration in Simulationsframeworks oder eigenständige Anwendungen.

```
reader = GeoJSONReader(EMBEDDED_GEOJSON)
geom_wgs84, geom_utm, to_utm, to_wgs84 = reader.load()

pv_type = PVTypeSelector.select(interactive=interactive)
tractor = TractorParameters.from_input(interactive=interactive)

wendestreifen = tractor.compute_wendestreifen_width()
fahrspur = tractor.compute_fahrspur_width()

# Margin: wir nehmen als konservativen Rand die größere der beiden Breiten
margin = max(wendestreifen, fahrspur)

usable_poly_utm, usable_area_m2 = AreaCalculator.compute_usable_area(geom_utm, margin)

grid_gen = PVGridGenerator(usable_poly_utm, to_wgs84)
row_spacing = fahrspur + 2.0 # grobe Heuristik: Fahrspur + 2m frei
geojson_points = grid_gen.generate_points(pv_type=pv_type, row_spacing_m=row_spacing, module_spacing_m=1.0)
```

2.4 Industriegebiet

module_industrial_area.py

Das Modul *module_industrial_area.py* implementiert eine modulare und objektorientierte Verarbeitungspipeline zur Aggregation von Lastgängen ausgewählter Unternehmen innerhalb eines Industriegebiets. Es dient als eigenständiges Werkzeug zur parametrisierten Auswahl von Unternehmen, zum Laden und Vorverarbeiten historischer Lastgangdaten sowie zur Erstellung von kombinierten CSV- und grafischen Darstellungen der Gesamtlast über Tages- oder Jahreszeiträume. Damit ermöglicht das Modul eine konsistente, reproduzierbare und automatisierbare Analyse industrieller Stromverbrauchsmuster.

Zu Beginn definiert das Modul eine Reihe verfügbarer Unternehmen (*UNTERNEHMEN*) inklusive typischer Betriebsparameter wie Wochenarbeitszeit, Schichtbetrieb und Referenzunternehmen. Die interne Zuordnung (*ZUORDNUNG*) verbindet diese Unternehmen mit den entsprechenden Spaltennamen der CSV-Daten und dem zugehörigen Jahr, sodass die Auswahl durch den Nutzer direkt auf die korrekten Lastgänge abgebildet wird.

```
# Liste der verfügbaren Unternehmen (Anzeige und Schlüssel zugleich)
UNTERNEHMEN: List[str] = [
    "Entsorgung: \n5 1/2 Tage Woche, Kein Schichtbetrieb, Anlehnung an Schönackers Umweltdienste\n",
    "Stahl: \n5 Tage Woche, 3-Schicht-Betrieb, Anlehnung an TecPro\n",
    "Logistik und Lagerung: \n5 Tage Woche, Teilweise Samstag und Sonntag Betrieb, Kein Schichtbetrieb,
    "Schrott- und Metallhandel: \n5 Tage Woche, Kein Schichtbetrieb, Anlehnung an Willi Jenner\n",
    "Lagerung: \n5 Tage Woche, Kein Schichtbetrieb, Anlehnung an Kleine Logistik\n",
    "Büro: \n5 Tage Woche, Kein Schichtbetrieb, Anlehnung an Marvin Wickenhäuser\n",
    "Logistik Lebensmittel: \n6 1/2 Tage Woche, Kein Schichtbetrieb, Anlehnung an CTJ Janssen\n",
    "Stahl: \n7 Tage Woche, 2-Schicht-Betrieb, Anlehnung an TecPro\n",
]
```

Die Klasse *IndustryLoadAggregator* kapselt die gesamte Logik für Interaktion, Datenverarbeitung, Aggregation und Ausgabe. Sie kann sowohl interaktiv über *run_interactive()* als auch programmgesteuert über *run(mode_choice, selection_indices)* verwendet werden. Über die Moduswahl kann zwischen Basisdaten ohne Wärme/Kälteverbrauch und erweiterten Daten mit Wärme- und Kälteszenarien gewählt werden, was die Flexibilität für unterschiedliche Szenarienanalysen erhöht.

Die Daten werden über die Methode *load_data()* aus CSV-Dateien geladen. Das Modul unterstützt hierbei unterschiedliche Quellpfade für Standardlastgänge und Lastgänge inklusive Wärme/Kälte. Nach dem Laden werden die Daten in einem

DataFrame für das Jahr 2017 gespeichert, was die Grundlage für die nachfolgende Aggregation bildet.

Die Auswahl der zu betrachtenden Unternehmen erfolgt über ein Eingabesystem oder programmgesteuerte Indizes. Die interne Methode *_parse_selection()* validiert Eingaben, begrenzt die maximale Anzahl ausgewählter Unternehmen auf fünf und gibt die entsprechenden Indizes zurück. Daraufhin werden die einzelnen Lastgänge in *aggregate()* extrahiert, mit eindeutigen Schlüsseln versehen, zusammengeführt und summiert. Das Ergebnis ist ein *DataFrame* mit der Gesamtlast (Gesamtlast [kW]) für jeden Zeitstempel, der für weitere Auswertungen genutzt werden kann.

```
for idx in selection_indices:
    if 0 <= idx < len(self.UNTERNEHMEN):
        u = self.UNTERNEHMEN[idx]
        unternehmen_counts[u] += 1
        key = f"{u}_{unternehmen_counts[u]}"
        lg, year = self.ZUORDNUNG[u]
        if year == "2017":
            df = self.df_2017[["Time stamp", lg]].copy()
            df.rename(columns={lg: key}, inplace=True)
            df[key] = pd.to_numeric(df[key], errors="coerce")
            selected_data[key] = df

if not selected_data:
    return None

combined_df = pd.DataFrame()
```

Für die Ausgabe stellt die Klasse zwei Methoden bereit: *save_csv()* exportiert die aggregierten Lastgänge als CSV-Datei, inklusive Zeitstempel im lesbaren Format, und erstellt bei Bedarf automatisch den Ausgabeordner. *plot_daily_bar()* erzeugt eine grafische Darstellung der täglichen Gesamtlast als Balkendiagramm, aggregiert über 24 Stunden, und formatiert die x-Achse nach Monaten. Beide Methoden geben die Pfade zu den erstellten Dateien zurück, sodass sie in automatisierten Workflows weiterverwendet werden können.

Das Modul ist so konzipiert, dass es reproduzierbare und konsistente Analysen industrieller Stromverbrauchsdaten ermöglicht. Die Trennung von interaktiver und programmgesteuerter Nutzung erlaubt sowohl explorative Analysen durch den Anwender als auch automatisierte Batch-Analysen für Simulationen oder Sensitivitätsstudien. Durch die klare Strukturierung in Auswahl, Aggregation und Ausgabe werden Fehlerquellen minimiert und die Nachvollziehbarkeit der erzeugten Daten sichergestellt.

2.5 Green Energy Hub

MA_Saur_GEH.py, electrolyzer_GEH.py

Die Generierung der Lastgänge für den Green Energy Hub erfolgt auf Basis des in der Masterarbeit von G. L. Saur entwickelten Simulationsansatzes zur Modellierung von Wasserstofftankstellen mit lokaler H_2 -Erzeugung. Der zugrunde liegende Python-Code wurde in das eigene, klassenbasierte Simulationsframework integriert und dient als Grundlage für die Abbildung der Wasserstoff- und Elektromobilitätsnachfrage.

[Saur, Gerrit L.: *Design einer Wasserstofftankstelle mit lokaler H_2 -Produktion aus erneuerbaren Energien am Beispiel des Green Energy Hub im Projekt Innovationspark Erneuerbare Energien Jüchen*. Masterarbeit, Technische Hochschule Köln, 2024.]

Die Umsetzung erfolgt als stochastische, ereignisbasierte Simulation. Einzelne Tank- und Ladevorgänge werden als diskrete Ereignisse modelliert und zu einer Zeitreihe mit viertelstündlicher Auflösung aggregiert. Die zeitliche Verteilung sowie die pro Vorgang abgegebene Energiemenge werden dabei aus geeigneten Wahrscheinlichkeitsverteilungen abgeleitet.

Die Gesamtanzahl der Ereignisse wird über die jährliche Nachfrage an Wasserstoff und elektrischer Energie skaliert, welche auf Basis von Fahrzeugbeständen, Fahrleistungen und spezifischen Verbräuchen bestimmt und auf eine einzelne Station heruntergebrochen wird.

Für die Wasserstoffnachfrage werden Pkw (FCEV) und Nutzfahrzeuge (FCET) berücksichtigt, während die elektrische Nachfrage batterieelektrischer Fahrzeuge über entsprechende Ladeprofile abgebildet wird. Die resultierenden Teilzeitreihen werden zu einer aggregierten elektrischen Last zusammengeführt und in das Gesamtsystem integriert.

2.6 Batterie

module_battery_storage.py

Zur Abbildung eines elektrischen Speichers wird im Modell ein vereinfachtes Batteriespeichermodul verwendet. Es verarbeitet die zuvor berechnete elektrische Bilanz des Gesamtsystems in zeitlich aufgelöster Form und simuliert, wie Überschüsse zwischengespeichert und Defizite durch Entladung teilweise ausgeglichen werden können.

Grundlage ist eine Bilanzzeitreihe in kW. Positive Bilanzwerte entsprechen einem Stromüberschuss, negative Werte einem Stromdefizit. Für jeden Zeitschritt wird geprüft, ob die Batterie geladen oder entladen werden kann. Dabei werden die Speicherkapazität, der aktuelle Ladezustand, die Lade- und Entladewirkungsgrade sowie die maximal zulässige Lade- und Entladeleistung berücksichtigt.

Bei einem Überschuss wird die Batterie geladen, sofern noch freie Speicherkapazität vorhanden ist. Bei einem Defizit wird die Batterie entladen, sofern ausreichend Energie im Speicher verfügbar ist. Kann ein Überschuss oder Defizit nicht vollständig durch den Speicher ausgeglichen werden, verbleibt der Rest in der Bilanz und wird weiterhin als Netzeinspeisung oder Netzbezug berücksichtigt.

Als Eingangsgrößen dienen insbesondere die Speicherkapazität, der Anfangsladezustand, der Wirkungsgrad, die C-Rate sowie die Bilanzzeitreihe des Systems. Als Ausgabe liefert das Modell eine neue Bilanzzeitreihe nach Speichereinsatz sowie den zeitlichen Verlauf des Ladezustands und der Batterieleistung. Zusätzlich werden zusammenfassende Kennwerte wie geladene und entladene Energie, Netzbezug, Netzeinspeisung sowie maximale Lade- und Entladeleistung bestimmt.

Das Batteriemodell dient damit dazu, den Einfluss eines Speichers auf die elektrische Bilanz des Gesamtsystems transparent und nachvollziehbar abzubilden. Es handelt sich um ein vereinfachtes, regelbasiertes Modell ohne Optimierung oder Alterungseffekte.

Klassenaufruf und Storage Objekt Erzeugung:

```
# When Battery input is given (capacity), battery class is called and storage object created
if inp.bs_energy_kwh is not None and inp.bs_energy_kwh > 0:
    storage = BatteryStorage(
        capacity_kwh=inp.bs_energy_kwh,           # Battery capacity [kWh]
        initial_soc_kwh=0,                         # Initial State of Charge [kWh]
        charge_efficiency=np.sqrt(inp.bs_efficiency), # Charging Efficiency
        discharge_efficiency=np.sqrt(inp.bs_efficiency), # Discharging Efficiency
        max_charge_c=inp.bs_c_rate,               # Maximum C-Rate Charging
        max_discharge_c=inp.bs_c_rate,            # Maximum C-Rate Discharging
        # Calculates max Charging/Discharging Power through capacity and C-Rate
    )
```

Berechnung der Batterie Ergebnisse:

```
# Calculate new energy balance (Data Frame['new_balance_kw'])
# Input: Balance Data Frame and give timestep information, 0.25 equals 15 min
# Output: Data Frame including: balance_kw(Input), battery_power_kw, soc_kwh, new_balance_kw
out.bs_df = storage.simulate(out.res_df["Bilanz [kW]"], timestep_hours=0.25)

# Calculate battery stats (dict)
# Input: result Battery Data Frame and give timestep information, 0.25 equals 15 min
# Output: Stats Dictionary: grid_import_kwh, grid_export_kwh, battery_charge_kwh, battery_discharge_kwh,
# ...final_soc_kwh, final_soc_percent, max_charge_kw, max_discharge_kw
out.bs_stats = storage.get_stats(out.bs_df, timestep_hours=0.25)
```

Beispiel Ergebnisse:

Batterie Data Frame:

	balance_kw	battery_power_kw	soc_kwh	new_balance_kw
0	-129.851870	0.0	0.0	-129.851870
1	-123.379070	-0.0	0.0	-123.379070
2	-136.365360	-0.0	0.0	-136.365360
3	-129.057590	-0.0	0.0	-129.057590
4	-589.365275	-0.0	0.0	-589.365275

Stats:

```
{'grid_import_kwh': 1428498.0, 'grid_export_kwh': 21886441.0, 'battery_charge_kwh': 3819781.0, 'battery_discharge_kwh': 3819781.0, 'final_soc_kwh': 0.0, 'final_soc_percent': 0.0, 'max_charge_kw': 117500.0, 'max_discharge_kw': 117500.0}
```

2.7 Input/Output Types

io_types.py

Zur strukturierten Kommunikation zwischen grafischer Benutzeroberfläche und Simulationsbackend werden zwei Datenstrukturen für Eingangsparameter und Simulationsergebnisse definiert.

Die Eingangsstruktur bündelt sämtliche vom Nutzer gewählten Szenarioparameter. Diese umfassen Energieerzeugung entlang von Autobahnen und Energielandschaften, Energieverbrauch von Industrieflächen und Green Energy Hubs sowie die Parametrisierung von Batteriespeichern. Durch optionale Eingaben können einzelne Module flexibel aktiviert oder deaktiviert werden, wodurch eine modulare Szenariokonfiguration ermöglicht wird.

Die Ausgabestruktur enthält sämtliche Simulationsergebnisse in aggregierter Form. Dazu gehören jährliche Energiemengen, maximale Leistungen der einzelnen Module sowie zeitaufgelöste Last- und Erzeugungsprofile. Zusätzlich werden Kennwerte zur Systembilanz, Eigenverbrauch und Autarkie bereitgestellt. Ergänzende Fehlermeldungen und Debug-Informationen ermöglichen eine robuste Ergebnisdarstellung und erleichtern die Analyse der Simulation.

Beispiel Input Objekt:

```
GuiInput  (h_highways          =['A44', 'A46'],
           h_pv_types          =[0, 2],
           el_build_state       =100,
           el_pv_type           =5,
           el_tractor_width     =12.0,
           el_tractor_length    =15.0,
           el_tractor_turn_radius =9.0,
           i_heat_cooling_electric =False,
           i_companies          =[0, 5],
           geh_h2_load_scenario ='2022',
           geh_e_load_scenario  ='2022',
           bs_efficiency         =1.0,
           bs_energy_kwh        =10000,
           bs_c_rate            =1.0,
           js_scenario          ='1000 Bewohner')
```

Beispiel Output Objekt:

```
SimOutput (h_sum_energy_kwh      =6244240.2450226145,
           el_sum_energy_kwh     =6295763.399954574,
           geh_sum_energy_kwh    =3398938.853155828,
           i_sum_energy_kwh      =6628304.2250149995,
           js_sum_energy_kwh     =1634962.7000000002,
           h_max_power_kw        =6973.651989068677,
           el_max_power_kw       =11658.527203199226,
           geh_max_power_kw      =11715.8288877,
           i_max_power_kw        =3311.9252500000002
           js_max_power_kw       =558.9, res_df=Data Frame,
           bs_df                 =Data Frame,
           bs_stats              =Dictionary,
           bal_energy_kwh        =877797.8668063604,
           bal_coverage_rate     =1.0,
           al_power_max_kw       =16182.820123222718,
           bal_power_min_kw      =-10995.754629685716,
           ee_self_consumption   =0.554232549819391,
           autarky               =0.5959488562540403,
           h_error_text          =",
           el_error_text         =None,
           geh_error_text        =None,
           i_error_text          ="Hier könnte ihre Industriegebiet
                                   Fehlermeldung stehen!",
           js_error_text         =",
           h_number_of_points    =48,
           el_pv_length          =48787.800000000002))
```

2.8 Graphical User Interface

Web_gui.py

Das Modul *Web_gui.py* implementiert die grafische Benutzeroberfläche des Simulationsframeworks auf Basis von *Streamlit* und fungiert als zentrale Schnittstelle zwischen Nutzereingabe, Simulationskern und Ergebnisvisualisierung. Die GUI ermöglicht eine parametrisierte Konfiguration der Teilmodelle ohne direkte Interaktion mit dem Quellcode und kapselt die Komplexität der Backend-Logik in einer interaktiven Webanwendung.

Zur lokalen Ausführung der grafischen Benutzeroberfläche wird die *Streamlit*-Anwendung über die Konsole gestartet. Dies erfolgt durch Aufruf der Datei *Web_gui.py* mit folgendem Befehl:

- `streamlit run <Pfad_zur_Datei>/ Web_gui.py`
- Dabei ist `<Pfad_zur_Datei>` durch den lokalen Speicherort der Datei zu ersetzen.

Zu Beginn werden alle benötigten Bibliotheken sowie projektspezifische Module importiert. Neben *Streamlit* für die UI-Struktur kommen Folium zur geographischen Visualisierung sowie *Matplotlib* für Zeitreihenplots zum Einsatz. Für die Darstellung von Energieflüssen wird zusätzlich *Plotly* verwendet, um interaktive Visualisierungen zu ermöglichen. Die Simulation wird über die zentrale Funktion *run_simulation()* aus dem Modul *Simulation* angesteuert, während Visualisierungen über Funktionen aus *plot_functions* und *web_support_functions* erzeugt werden.

Zur Überführung von GUI-Auswahloptionen in modellinterne Parameter werden *Mapping-Dictionaries* definiert (z. B. *pv_name_to_id*, *industry_csv_mapping*). Diese ermöglichen eine saubere Trennung zwischen benutzerfreundlicher Darstellung und backendseitiger Datenstruktur.

Die Kommunikation zwischen GUI und Simulationskern erfolgt über die Klassen *GuiInput* und *SimOutput*. Während *GuiInput* sämtliche Eingabeparameter kapselt, dient *SimOutput* als strukturierter Container für Simulationsergebnisse. Über *st.session_state* wird ein persistenter Zustand realisiert, sodass Simulationsergebnisse über mehrere Interaktionen hinweg verfügbar bleiben.

```
Input = GuiInput()
Output = SimOutput()

if "simulation_ran" not in st.session_state:
    st.session_state.simulation_ran = False

if "sim_output" not in st.session_state:
    st.session_state.sim_output = Output
```

Die Benutzeroberfläche ist modular aufgebaut und in die Bereiche Erzeugung, Verbrauch und Speicher gegliedert. Die Eingabe erfolgt über *Streamlit*-Widgets (z. B. *selectbox*, *multiselect*, *segmented_control*), deren Werte direkt in das *GuiInput*-Objekt geschrieben werden.

```
# Fester Container für Darstellung der Erzeugungseingabe
with st.container(key="Erzeugung", width=1000, height='content'):

    tab1, tab2 = st.tabs(["Solarautobahn", "Energiewirtschaft"])
```

Im Bereich der Erzeugung werden die Teilmodelle Solarautobahn und Energiewirtschaft über Tabs konfiguriert. Neben der Parametereingabe erfolgt eine dynamische Visualisierung der gewählten Standorte über *GeoJSON*-Daten mittels *Folium*. Dadurch wird eine direkte Kopplung zwischen Modellparametern und räumlicher Repräsentation hergestellt.

Die Verbrauchsmodelle (Industriegebiet, Green Energy Hub, Jüchen Süd) werden ebenfalls *tab*-basiert konfiguriert. Hierbei werden Lastprofile indirekt über Szenarien oder Auswahl diskreter Unternehmensprofile parametrisiert. Ergänzend werden Vorschauplots generiert, um die gewählten Lastgänge qualitativ zu verifizieren.

Die Parametrisierung des Batteriespeichers erfolgt über Kapazität, Wirkungsgrad und C-Rate. Die GUI übernimmt hierbei ausschließlich die Konfiguration, während die physikalische Modellierung im Backend erfolgt.

Die Simulation wird über einen zentralen Trigger (*st.button*) gestartet. Dabei wird das vollständig befüllte *GuiInput*-Objekt an *run_simulation(Input)* übergeben. Die Funktion führt die Berechnung der Energieflüsse durch und gibt ein *SimOutput*-Objekt zurück, das sämtliche Zeitreihen und Bilanzgrößen enthält.

```
with col2:
    run = st.button(
        label="⚡ Simulation starten",
        type="primary",
        use_container_width=True
    )
st.space(size='small')

if run:
    with st.spinner("Simulation läuft..."):
        Output = run_simulation(Input)

    st.success("Simulation abgeschlossen.")
    st.session_state.simulation_ran = True
    st.session_state.sim_output = Output
```

Die Ergebnisdarstellung umfasst mehrere Ebenen: Zunächst wird ein Synergie-Diagramm zur Visualisierung der Energieflüsse erzeugt, welches mithilfe von *Plotly* als interaktive Darstellung umgesetzt wird. Anschließend werden zeitaufgelöste Bilanzverläufe sowie statistische Kennwerte dargestellt. Bei aktivem Batteriespeicher werden zusätzlich Zustandsgrößen wie Ladezustand und Speicherleistung ausgewertet.

Die *WebGUI.py* bildet somit eine integrale Systemkomponente, die Eingabe, Simulation und Auswertung in einem konsistenten Workflow verbindet und die modulare Struktur des Gesamtsystems auf Benutzerebene abbildet.

2.8.1 Unterstützende Webfunktionen

web_support_functions.py

Ergänzend zur Hauptdatei *Web_gui.py* werden in *web_support_functions.py* Hilfsfunktionen für die visuelle Aufbereitung der GUI-Ergebnisse bereitgestellt. Das Modul übernimmt die Vorbereitung von Icons, die Formatierung von Energie-, Leistungs- und Kapazitätswerten sowie die Konstruktion des interaktiven Synergie-Diagramms auf Basis von *Plotly*.

Zu Beginn des Moduls werden die Pfade der in der Webanwendung verwendeten Icons definiert (*ICON_PATHS*), welche die einzelnen Teilsysteme wie Solarautobahn, Energielandschaft, Batteriespeicher, Industriegebiet, Green Energy Hub, Stromnetz und Jüchen Süd repräsentieren. Zusätzlich werden Skalierungsfaktoren (*ICON_SCALE*) für die Symbolgrößen festgelegt, um eine konsistente Darstellung im Diagramm sicherzustellen. Für die Einbindung der Logos und Icons in HTML- bzw.

Streamlit-Komponenten wird außerdem eine Funktion zur Base64-Kodierung von Bilddateien bereitgestellt.

Ein weiterer Bestandteil des Moduls ist die Formatierung numerischer Ergebnisgrößen. Die Funktionen `format_energy()`, `format_power()` und `format_capacity()` überführen Rohwerte aus der Simulation in ein nutzerfreundliches Anzeigeformat, sodass Energie-, Leistungs- und Kapazitätswerte in geeigneten Einheiten dargestellt werden. Dies verbessert insbesondere die Lesbarkeit von KPI-Anzeigen und Hover-Informationen.

Für die grafische Darstellung des interaktiven Synergie-Diagramms werden mehrere Hilfsfunktionen verwendet, die wiederkehrende visuelle Elemente miteinander verbinden:

- Die Funktion `add_node_with_icon()` fügt einem *Plotly*-Diagramm einen Systemknoten mit Symbol und Beschriftung hinzu. Damit werden alle Hauptkomponenten des Energiesystems einheitlich dargestellt.
- Die gerichteten Energieflüsse zwischen den Knoten werden durch `add_routed_arrow()` erzeugt. Diese Funktion zeichnet Pfeile entlang mehrerer Zwischenpunkte und erlaubt dadurch eine saubere Linienführung mit Richtungswechseln. So können die Flüsse im Diagramm übersichtlich angeordnet werden, ohne dass sich Verbindungen unnötig überlagern.
- Damit große und kleine Energieströme im Diagramm visuell unterscheidbar bleiben, wird die Pfeildicke in Abhängigkeit vom jeweiligen Flusswert skaliert. Dies geschieht über `scale_arrow_width()`. Der größte im Diagramm vorkommende Energiefluss dient dabei als Referenzwert.

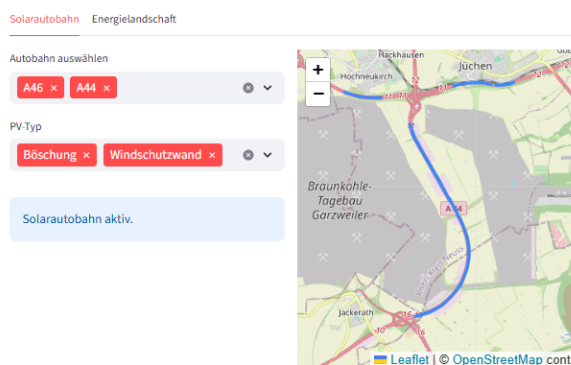
Die Hauptfunktion `build_synergy_diagram(inp, sim_output)` verarbeitet die im *SimOutput*-Objekt enthaltenen Ergebnisse und erzeugt daraus eine interaktive *Plotly*-Grafik. Sie übernimmt die in der Simulation berechneten Ergebnisse und übersetzt diese in eine interaktive Systemgrafik. Die Funktion greift auf das *SimOutput*-Objekt zu und liest daraus alle relevanten Jahresenergiemengen, Batteriegrößen und Bilanzkennwerte aus. Dazu zählen insbesondere die Erzeugung der Solarautobahn und Energielandschaft, die Lasten des Industriegebiets, des Green Energy Hub und von Jüchen Süd sowie Netzbezug, Netzeinspeisung und batterierelevante Kenngrößen.

Im nächsten Schritt wird aus diesen Werten die bilanzielle Logik der Darstellung aufgebaut. Hierzu werden Gesamterzeugung und Gesamtbedarf berechnet und ein möglicher Überschuss (*surplus*) oder ein Defizit (*deficit*) bestimmt. Diese Logik entscheidet später darüber, ob im Diagramm ein Pfeil zur Netzeinspeisung oder ein Pfeil für Netzbezug dargestellt wird. Die Flussbreiten werden entsprechend der Jahresenergiemengen skaliert.

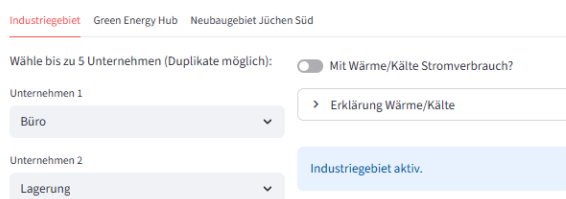
`web_support_functions.py` übernimmt somit die visuelle Interpretation der Simulationsergebnisse und ergänzt `Web_gui.py` um die für die Ergebnisdarstellung erforderlichen Darstellungsfunktionen.

Energiesystem Modellierung - Innovationspark Jüchen

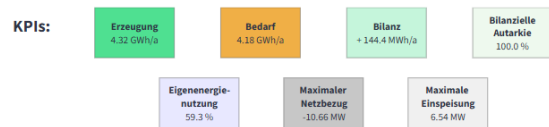
Erzeugung Eingabewerte



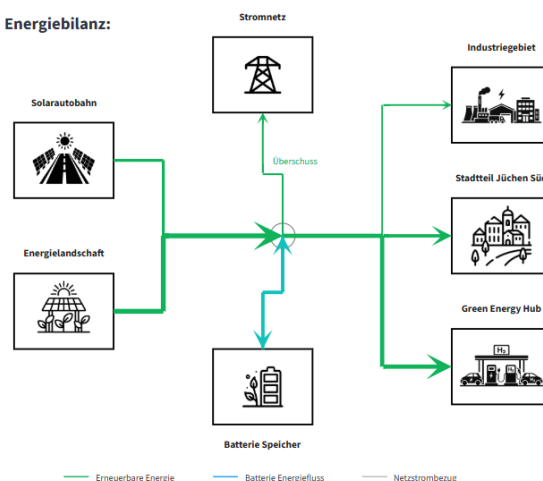
Verbrauch Eingabewerte



Ergebnisse



Energiebilanz:



2.9 Simulation

`simulation.py`

Das Modul *simulation.py* bildet den zentralen Simulationskern des Energiesystem-Modellierungsframeworks. Es übernimmt die Integration sämtlicher Teilmodelle, führt die Berechnung der Energieflüsse durch und stellt die Simulationsergebnisse in strukturierter Form zur weiteren Verarbeitung und Visualisierung bereit. Die Simulation kombiniert dabei mehrere unabhängige Erzeugungs- und Verbrauchsmodelle sowie optional einen Batteriespeicher, um ein ganzheitliches Energiesystem abzubilden.

Zu Beginn des Moduls werden alle benötigten Bibliotheken sowie projektspezifische Module importiert. Neben Standardbibliotheken wie *NumPy* und *Pandas* werden insbesondere die projektspezifischen Komponenten zur Modellierung der einzelnen Teilsysteme geladen. Hierzu zählen unter anderem das PV-Modul, das Industriegebietsmodell, die Energielandschaft, der Batteriespeicher sowie die Datentypen für Ein- und Ausgabeobjekte. Diese Struktur erlaubt eine modulare Erweiterung des Simulationsframeworks, da zusätzliche Komponenten problemlos ergänzt werden können.

Zur Abbildung benutzerseitiger Auswahloptionen werden im Anschluss mehrere *Mapping-Dictionaries* definiert. Diese dienen der Übersetzung von GUI-Eingaben in modellinterne Parameter und ermöglichen somit eine saubere Trennung zwischen Benutzeroberfläche und Simulationslogik. Beispielsweise werden Unternehmensauswahlen, Autobahnabschnitte oder PV-Systemtypen über Mapping-Strukturen in interne Modellparameter überführt. Dadurch bleibt die Simulation unabhängig von der konkreten GUI-Implementierung und kann flexibel angepasst werden.

Die zentrale Simulationslogik ist in der Klasse *Simulator* implementiert. Diese Klasse kapselt sämtliche Teilmodelle und stellt Funktionen zur Durchführung der einzelnen Simulationen bereit. Beim Initialisieren des *Simulators* werden zunächst die Wetterdaten geladen und aufbereitet. Die Wetterdaten dienen als gemeinsame Datengrundlage für alle photovoltaischen Simulationen und werden daher nur einmal eingelesen, um redundante Berechnungen zu vermeiden. Die Funktion zur Aufbereitung der Wetterdaten liest die entsprechenden CSV-Dateien ein, konvertiert Zeitstempel und stellt die Daten in einheitlicher Form für alle Module bereit.

```
def weather_prep(self): 1 usage  @jbrinkma
    """
    Prepares the weather data used for all PV simulations.
    """
    weather_df = pd.read_csv(weather_csv, index_col=0, parse_dates=True)
    weather_df.index = pd.to_datetime(weather_df.index, utc=True).tz_convert(None)

    return weather_df
```

Das erste Teilmodell innerhalb der Simulation ist die Solarautobahn. Die Funktion `run_pv_modulator()` generiert hierbei Photovoltaik-Installationspunkte entlang ausgewählter Autobahngeometrien. Anschließend wird für jeden ausgewählten PV-Typ die elektrische Leistung simuliert und zu einer Gesamterzeugung aggregiert. Die Simulation berücksichtigt dabei die Schrittweite der Modulplatzierung sowie die geometrischen Eigenschaften der PV-Systeme. Das Ergebnis ist eine zeitaufgelöste Erzeugungsreihe sowie Kennwerte wie maximale Leistung und Jahresenergieertrag.

```
# highways are simulated
for highway in inp.h_highways:
    selected_file = highway_geojson_mapping[highway]
    geojson_path = os.path.join(base_input, "geometry", selected_file)

    h_pv_system = PV_System(
        geojson_path=geojson_path,
        pvtypes=h_selected_pv_types,
        step_size=800,
        weather_df=self.weather_df,
        base_output_dir=os.path.join(output_dir, "highway"),
    )

    results = h_pv_system.generate_pv_points()
```

Das zweite Erzeugungsmodell bildet die Energielandschaft bzw. Agri-PV-Installation ab. Die Funktion `run_energy_landscape()` generiert dabei PV-Strukturen auf landwirtschaftlichen Flächen unter Berücksichtigung von Maschinenparametern wie Traktorbreite, Wendekreis und Reihenabstand. Basierend auf diesen Parametern wird eine geeignete PV-Anordnung erstellt und anschließend die Energieerzeugung simuliert. Zusätzlich wird ein Ausbaugrad berücksichtigt, um unterschiedliche Szenarien abbilden zu können.

Neben den Erzeugungsmodellen werden mehrere Verbrauchsmodelle implementiert. Das Industriegebietsmodell wird über die Funktion `run_industry_area()` berechnet. Hierbei werden Lastprofile verschiedener Unternehmen aggregiert und optional eine Elektrifizierung von Heiz- und Kühlprozessen berücksichtigt. Das Ergebnis ist ein

kombinierter Lastgang für das gesamte Industriegebiet sowie entsprechende Kennwerte wie maximale Leistung und Gesamtenergiebedarf.

```
# Run industry_aggregator with user inputs
df_industrie, csv_path_industrie, plot_path_industrie = industry_aggregator.run(
    mode_choice=industry_heating_choice_mapping[inp.i_heat_cooling_electric],
    selection_indices=inp.i_companies,
)
# Write timeseries
if df_industrie is None or df_industrie.empty:
    out.res_df['Last_Industrie [kW]'] = 0
else:
    out.res_df['Last_Industrie [kW]'] = df_industrie['Gesamtlast [kW]'] # Alle Lastgänge positiv
```

Das Modul `run_green_energy_hub()` simuliert zusätzlich den Strombedarf eines Green Energy Hub. Hierbei werden Szenarien für Wasserstoffproduktion und elektrische Mobilität geladen und zu einem gemeinsamen Lastprofil kombiniert. Die Lastgänge werden aus CSV-Dateien eingelesen und zeitlich auf die Simulation abgestimmt. Das Ergebnis ist ein zeitaufgelöster Lastgang für den gesamten Green Energy Hub.

Als weiteres Verbrauchsmodell wird in `run_juechen_sued()` die Entwicklung eines neuen Wohngebiets berücksichtigt. Dabei wird ein vordefinierter Lastgang geladen und entsprechend der gewählten Einwohnerzahl skaliert. Auf diese Weise lassen sich unterschiedliche Ausbaustände der Stadtentwicklung simulieren.

Nachdem alle Erzeugungs- und Verbrauchsmodelle berechnet wurden, erfolgt in der Funktion `run_balance()` die Berechnung der Gesamtbilanz des Energiesystems. Hierbei werden sämtliche Erzeugungs- und Lastprofile miteinander verrechnet, sodass eine zeitaufgelöste Leistungsbilanz entsteht. Zusätzlich werden Kennwerte wie Gesamtenergie, Deckungsgrad sowie maximale Import- und Exportleistung bestimmt.

```
sign_map = {
    "Erzeugung_Solarautobahn [kW]": 1,
    "Erzeugung_Energie Landschaft [kW]": 1,
    "Last_Industrie [kW]": -1,
    "Last_Green_Energy_Hub [kW]": -1,
    "Last_Juechen_Sued [kW]": -1,
}

balance = 0

for col, sign in sign_map.items():
    if col in out.res_df.columns:
        balance = balance + out.res_df[col] * sign

out.res_df["Bilanz [kW]"] = balance
```

Optional kann anschließend ein Batteriespeicher in die Simulation integriert werden. Die Funktion `run_battery()` simuliert das Lade- und Entladeverhalten des Speichers auf Basis der zuvor berechneten Leistungsbilanz. Dabei werden Wirkungsgrad, Kapazität und C-Rate berücksichtigt. Die Simulation liefert den Ladezustand, die Speicherleistung sowie die angepasste Netzbilanz.

Die Hauptfunktion `run_simulation()` bildet schließlich den zentralen Einstiegspunkt des Simulationskerns. Diese Funktion ruft alle Teilmodelle sequentiell auf und kombiniert deren Ergebnisse zu einer Gesamtsimulation. Nach Durchführung der Simulation werden zusätzliche Kennwerte wie Eigenverbrauchsanteil und Autarkiegrad berechnet. Die Ergebnisse werden anschließend in einem `SimOutput`-Objekt zusammengefasst und zur weiteren Verarbeitung zurückgegeben.

```
sim.run_pv_modulator(inp, out)
print('Solarautobahn \t\t- abgeschlossen')

sim.run_energy_landscape(inp, out)
print('Energiewirtschaft \t- abgeschlossen')

sim.run_industry_area(inp, out)
print('Industriegebiet \t- abgeschlossen')

sim.run_green_energy_hub(inp, out)
print('Green Energy Hub \t- abgeschlossen')

sim.run_juechen_sued(inp, out)
print('Juechen-Sued \t- abgeschlossen')

total_demand, total_generation = sim.run_balance(out)

sim.run_battery(inp, out)
print('Batterie \t\t- abgeschlossen')
```

Der Simulationskern stellt somit die zentrale Rechenkomponente des Frameworks dar. Durch die modulare Struktur können einzelne Teilmodelle unabhängig voneinander entwickelt und erweitert werden. Gleichzeitig ermöglicht die gemeinsame Bilanzierung eine integrierte Bewertung komplexer Energiesysteme mit verschiedenen Erzeugungs-, Verbrauchs- und Speicherkomponenten.

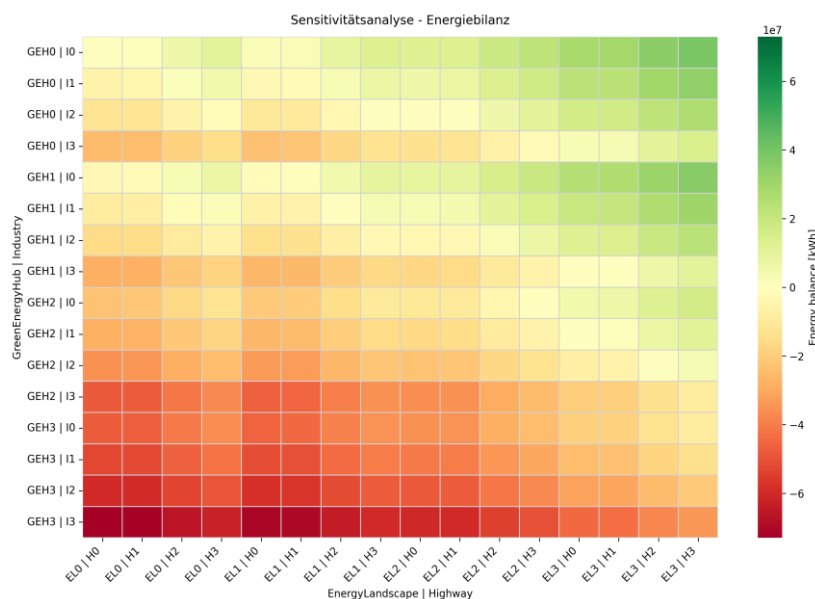
2.10 Sensitivitätsanalyse

sensitivity_analysis.py

Das Skript dient zur automatisierten Sensitivitätsanalyse des Energiesystemmodells. Hierfür werden vordefinierte Ausbaustufen der Teilbereiche Solarautobahn, Energielandschaft, Industriegebiet, Green Energy Hub und Batteriespeicher systematisch kombiniert und jeweils an die zentrale Simulationsfunktion übergeben. Grundlage dafür ist ein `GuiInput`-Objekt, das für jeden Simulationslauf aus den gewählten Szenariokonfigurationen aufgebaut wird.

Die eigentliche Analyse erfolgt in `run_sensitivity_analysis()`. Dort werden alle definierten Szenariokombinationen iterativ durchlaufen, simuliert und hinsichtlich ihrer Gesamtbilanz ausgewertet. Die Ergebnisse werden in einer Matrix zusammengeführt, deren Zeilen Verbrauchsszenarien und deren Spalten Erzeugungsszenarien repräsentieren. Falls ein Batteriespeicher berücksichtigt wird, wird die angepasste Bilanz aus den Speicherdaten verwendet.

Zusätzlich werden die zeitlich aufgelösten Simulationsergebnisse jeder Konfiguration als CSV-Dateien gespeichert. Die aggregierte Ergebnismatrix wird als Excel-Datei exportiert und anschließend als Heatmap visualisiert. Die Heatmap ermöglicht einen kompakten Vergleich der Szenariokombinationen und stellt positive sowie negative Bilanzwerte über eine um null zentrierte Farbskala dar. Das Ausgabeverzeichnis wird bei Bedarf automatisch erzeugt.



2.11 Hilfsfunktionen

Zusätzlich zur zentralen Simulationspipeline existieren mehrere eigenständige Module, die als Hilfsfunktionen dienen und optional ausgeführt werden können. Diese Module unterstützen die Datenaufbereitung, Visualisierung und Simulation einzelner Komponenten, ohne die Hauptlogik zu verändern. Sie lassen sich direkt über ein `__main__`-Interface aufrufen.

2.11.1 Geometrie-Utilities

`functions_geometrie.py`

Das Geometriemodul dient der Extraktion, richtungsbezogenen Filterung, Visualisierung und Speicherung von Autobahngeometrien als räumliche Eingangsgrößen für die PV-Simulation entlang der Solarautobahn. Hierzu werden OpenStreetMap-Daten mithilfe von *osmnx* geladen, auf Autobahnen (motorway) gefiltert und als *GeoDataFrames* weiterverarbeitet.

Die Klasse *DirectionUtilis* übernimmt die Orientierung einzelner Liniengeometrien. Neben der Anpassung an Einbahnrichtungen erfolgt die Berechnung von Richtungswinkeln (*Bearing*) sowie deren Zuordnung zu vorgegebenen Himmelsrichtungen unter Berücksichtigung einer Winkeltoleranz.

Die Klasse *HighwayExtractor* extrahiert relevante Autobahnsegmente innerhalb vordefinierter *Bounding Boxes*. Dabei werden die Geometrien mit den Untersuchungsbereichen geschnitten und anschließend anhand ihrer Fahrtrichtung gefiltert, sodass nur Segmente mit passender Orientierung in das Ergebnis eingehen.

Autobahn	Abschnitt	Bounding Box (lon/lat)
A46	1	(6.44730, 51.09131, 6.46292, 51.09533)
A46	2	(6.49446, 51.09495, 6.51060, 51.09845)
A46	3	(6.51206, 51.09495, 6.52021, 51.09786)
A44	1	(6.474917, 51.03384, 6.50147, 51.085915)

Die Klasse *GeometryHandler* dient der weiteren Verarbeitung der gefilterten *GeoDataFrames*. Sie ermöglicht die Visualisierung der Geometrien auf einer interaktiven Karte (Folium) sowie den Export als *GeoJSON*-Datei unter Beibehaltung relevanter Attribute wie Fahrtrichtung und Referenz.

Die Funktion `run_autobahn_geometrie()` stellt einen interaktiven Einstiegspunkt zur Auswahl, Darstellung und Speicherung definierter Autobahnabschnitte bereit. Das Modul bildet damit die räumliche Vorverarbeitung für die nachgelagerte Punkt- und Azimutberechnung im PV-Modul.

2.11.2 Wetterdatenmodul

`functions_weather_data.py`

Das Modul `functions_weather_data.py` dient der Bereitstellung und Aufbereitung der Wetterdaten für die PV-Simulation. Es basiert auf Beobachtungsdaten des Deutschen Wetterdienstes (DWD) und stellt die meteorologischen Eingangsgrößen in einer konsistenten und für `pvlb` nutzbaren Form bereit. Damit bildet das Modul die Datengrundlage für die Berechnung der PV-Erzeugung.

Die Konfiguration des Datenabrufs erfolgt über die Dataclass `DWDWeatherConfig`. In ihr werden unter anderem Koordinaten, Simulationszeitraum, Stationsauswahl, Qualitätsanforderungen, Zeitzone sowie optionale Speicher- und Plotparameter definiert. Zusätzlich können Parameter zur Berechnung der Direktnormalstrahlung (DNI) festgelegt werden, darunter die Nutzung des `dirint`-Verfahrens zur robusten Bestimmung der DNI aus Globalstrahlung und Sonnengeometrie, ein minimaler Sonnenhöhenwinkel sowie die zulässige Interpolation kurzer Datenlücken.

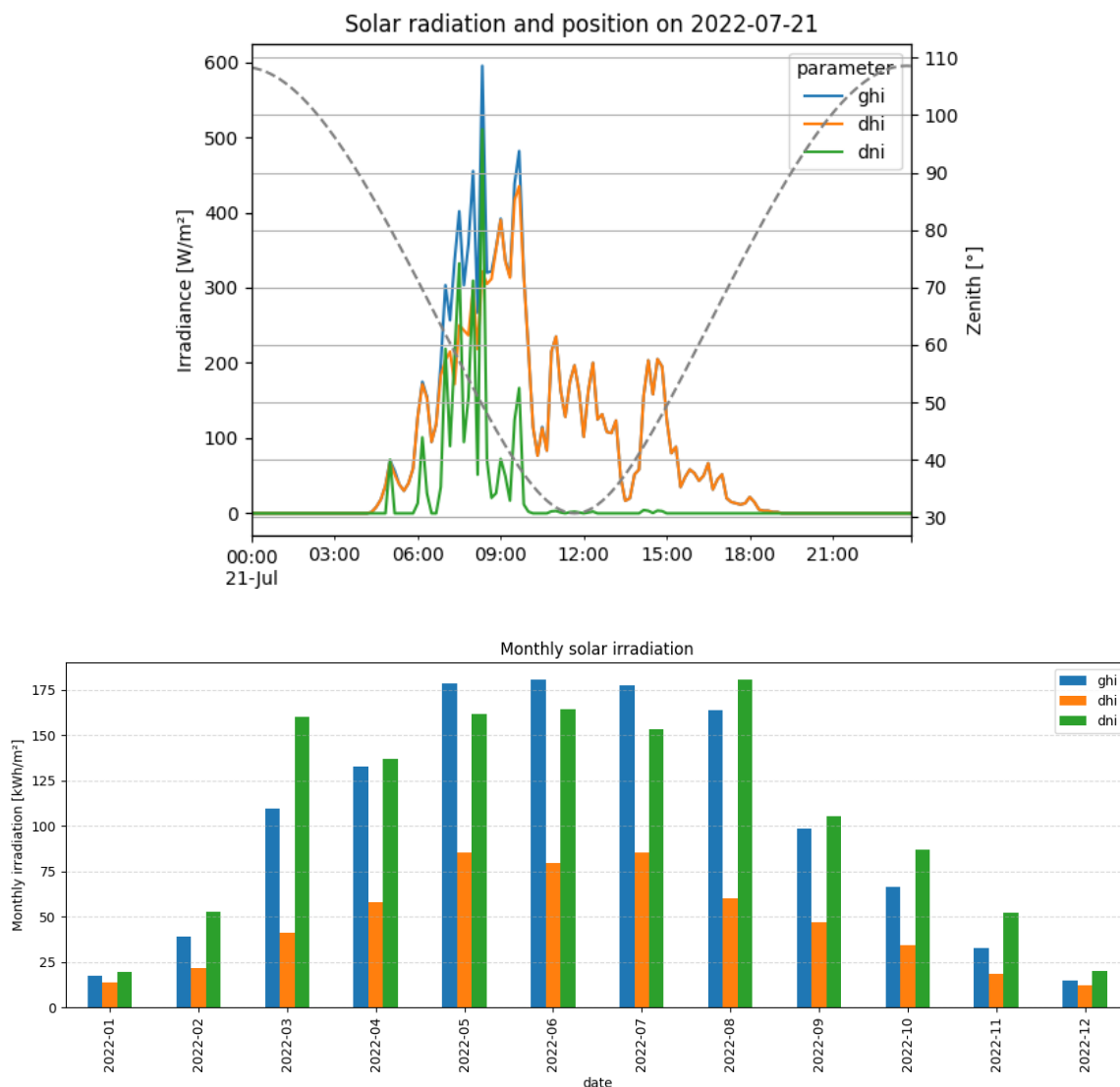
```
# --- DNI calculation ---  
if self.cfg.use_dirint:  
    dni_vals = dirint(  
        ghi=df["ghi"].values,  
        solar_zenith=df["zenith"].values,  
        times=df.index,
```

Die zentrale Verarbeitung erfolgt in der Klasse `DWDWeatherFetcher`. Die Methode `run()` umfasst die automatische Auswahl einer geeigneten Wetterstation, den Abruf der Rohdaten sowie deren Aufbereitung und optionale Speicherung.

Im Rahmen der Aufbereitung werden die Rohdaten in ein tabellarisches Format überführt, umbenannt und in geeignete Einheiten transformiert. Dazu zählen insbesondere die Umrechnung der Temperatur von Kelvin in Grad Celsius sowie der Strahlungsgrößen von J/m^2 pro 10 Minuten in W/m^2 . Anschließend werden die Daten auf ein konsistentes 10-Minuten-Zeitraster gebracht und interpoliert.

Zusätzlich wird auf Basis der Standortkoordinaten die Solarposition berechnet. Daraus werden Zenitwinkel und Sonnenhöhe abgeleitet, die für die Bestimmung der Direktnormalstrahlung erforderlich sind. Für die Berechnung der DNI stehen zwei Ansätze zur Verfügung: standardmäßig das *dirint*-Verfahren auf Basis von Globalstrahlung und Sonnengeometrie sowie alternativ eine klassische Inversion aus Global- und Diffusstrahlung. Zur Sicherstellung physikalisch plausibler Ergebnisse werden negative Werte ausgeschlossen, Nachtstunden auf null gesetzt und Obergrenzen durch extraterrestrische Strahlung berücksichtigt.

Das Ergebnis ist ein aufbereitetes Wetterdaten-*DataFrame* mit den für die PV-Simulation relevanten Größen, insbesondere *ghi*, *dhi*, *dni* und *temp_air*. Optional können die Daten als CSV-Datei gespeichert und mithilfe von Plotfunktionen visuell überprüft werden. Das Modul stellt damit eine konsistente und reproduzierbare Datengrundlage für die energetische Simulation bereit.



3 Annahmen Case Study Innovationspark Jüchen

Annahmen bezüglich der Teilprojekte der Case Study Innovationspark Jüchen

3.1 Solarautobahn

Die Ausgestaltung der Solarautobahn basiert auf den Ergebnissen der Machbarkeitsstudie des Innovationsparks Erneuerbare Energien Jüchen. Insbesondere die Auswahl der betrachteten PV-Konzepte (Lärmschutzwand-PV, Lärmschutzkassetten-PV, Windschutzwand-PV, Böschung-PV) sowie die räumliche Eingrenzung auf die Autobahnabschnitte A44 und A46 orientieren sich an dieser Studie. ([Machbarkeitsstudie Solarautobahn](#))

- Autobahnstrecke A44 und A46 im Bereich Tagebau Garzweiler (ca. 30 km)
- PV-Anlagen Konzepte:
 - Lärmschutzwand-PV
 - Lärmschutzkassetten-PV
 - Windschutzwand-PV
 - Böschung-PV
- Die energetische Bewertung basiert auf Wetterdaten des Deutschen Wetterdienstes ([DWD](#))
- Die zeitliche Auflösung der Daten entspricht der Modellauflösung (15 Min.)

3.2 Energielandschaft

Die Auslegung der PV-Konstruktionen orientiert sich an den technischen Empfehlungen des Fraunhofer ISE zur Agri-Photovoltaik ([Fraunhofer Agri-Photovoltaik](#)). Die Definition und Abgrenzung der betrachteten Flächen basiert auf der Machbarkeitsstudie „Energielandschaft“ des Innovationsparks Erneuerbare Energien Jüchen ([Machbarkeitsstudie Energielandschaft](#)).

Fläche für Agri-PV-Installation:

- Gesamtfläche für Energielandschaft anhand Machbarkeitsstudie definiert
- GeoJSON-Datei der Gesamtfläche mit geojson.io (<https://geojson.io/next/>) erstellt
- Anhand Traktorparameter (Länge, Breite, Wenderadius) wird Breite des Wendestreifens und der Fahrspur berechnet

Annahme PV-Module:

- Bifaziale Module
- Modul Abmessungen: 1m * 2m
- Sicherheitsabstand zusätzlich zu Fahrspur: 1m

PV-Anlagen:

- Geneigte PV-Module
 - kein Tracking
 - Ost-West Reihen
 - Südausrichtung
 - 2 Module übereinander: 4m
 - Neigung: 30°
 - Bodenbreite: $4\text{m} * \cos(30^\circ) = 3,5\text{m}$
 - Unterfahren: 1m (einseitig) → Traktor kann teilweise unter Module fahren
 - Reihenbreite: $3,5\text{m} - 1\text{m} = 2,5\text{m}$
- Geneigte PV-Module
 - mit Tracking
 - Nord-Süd Reihen
 - Ost-West Tracking
 - 4 Module übereinander: 4m

- Neigung: 0° (bei Tracking in Wartungsmodus)
 - Bodenbreite: 4m (Aufständigung deutlich schmaler)
 - Unterfahren: 1m (beidseitig, im Wartungsmodus) → Traktor kann teilweise unter Module fahren
 - Reihenbreite: 4m - 1m - 1m = 2m
- Vertikale PV-Module
 - Zaun
 - Nord-Süd Reihen
 - Ost/West Ausrichtung
 - 2m hohes Modul → jeweils 1 Modul
 - Reihenbreite: 0,5m

3.3 Green Energy Hub

Die Ausgestaltung des Green Energy Hub basiert auf den Ergebnissen der Machbarkeitsstudie des Innovationsparks Erneuerbare Energien Jüchen sowie auf der Masterarbeit von G. L. Saur zur Auslegung einer Wasserstofftankstelle mit lokaler H₂-Produktion. Insbesondere die Szenariendefinition, das Nutzerverhalten sowie die zeitliche Verteilung der Tankvorgänge orientieren sich an diesen Arbeiten und bilden die Grundlage für die Modellierung der Lastgänge und Energieflüsse innerhalb des Green Energy Hub. ([Machbarkeitsstudie GEH](#)), (Saur)

- Schnittstelle zwischen Energieerzeugung und Mobilitätssektor
- Bereitstellung von Infrastruktur für:
 - Wasserstoff (H₂-Tankstelle)
 - Elektrisches Laden (BEV-Ladeinfrastruktur)
 - 10 MWh Wasserstoffspeicher
- Szenariendefinition für die Jahre:
 - 2022
 - 2030
 - 2050

Tabelle: Wasserstoffmarktdiffusion ausgewählter Märkte für 2050 (Saur)

Szenario	PKW	LKW	Bus
Gering	25%	25%	30%
Mittel	50%	50%	60%
Hoch	75%	75%	95%

- Die Skalierung der Wasserstoffnachfrage erfolgt anhand prognostizierter Fahrzeugzahlen:
 - Entwicklung der Anzahl an FCEVs bis 2050 gemäß Literatur (Saur)
 - Differenzierung nach Fahrzeugklassen (PKW, LKW, Busse)

Tabelle: Prognosen zur Anzahl an FCEVs (nach Saur)

Jahr	FCEVs	PKW	LKW	Busse
2020	10.000	9.277	707	16
2025	100.000	92.773	7.067	160
2030	1.800.000	1.669.914	127.206	2.880
2050	4.560.000	4.230.449	322.255	7.296

Die Modellierung der Wasserstoffnachfrage erfolgt über stochastisch generierte Lastgänge: (Saur)

- Basierend auf stündlichen Tankwahrscheinlichkeiten in Abhängigkeit von Tageszeit und Wochentag
- Überführung in eine zeitliche Auflösung von 15 Minuten entsprechend der Gesamtmodellierung
- Kombination mit fahrzeugspezifischen Verbrauchsannahmen (FCEV, Nutzfahrzeuge)
- Das Nutzerverhalten (Tankzeitpunkte und -mengen) wird auf Basis, der in der Masterarbeit von Saur entwickelten und validierten Modelle abgebildet
- Dabei werden zeitliche Wahrscheinlichkeitsverteilungen sowie Verteilungen der Tankmengen zur Generierung einzelner Tankereignisse verwendet

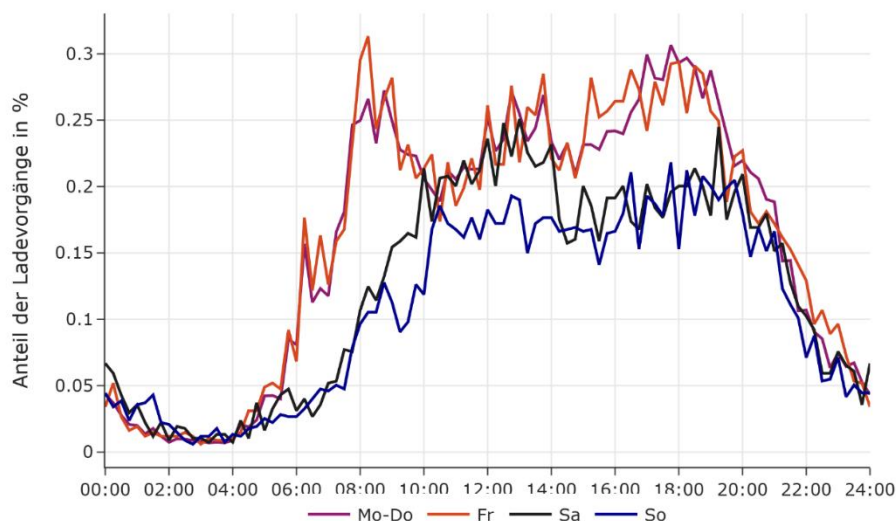


Abbildung: Relative Ladewahrscheinlichkeit BEV

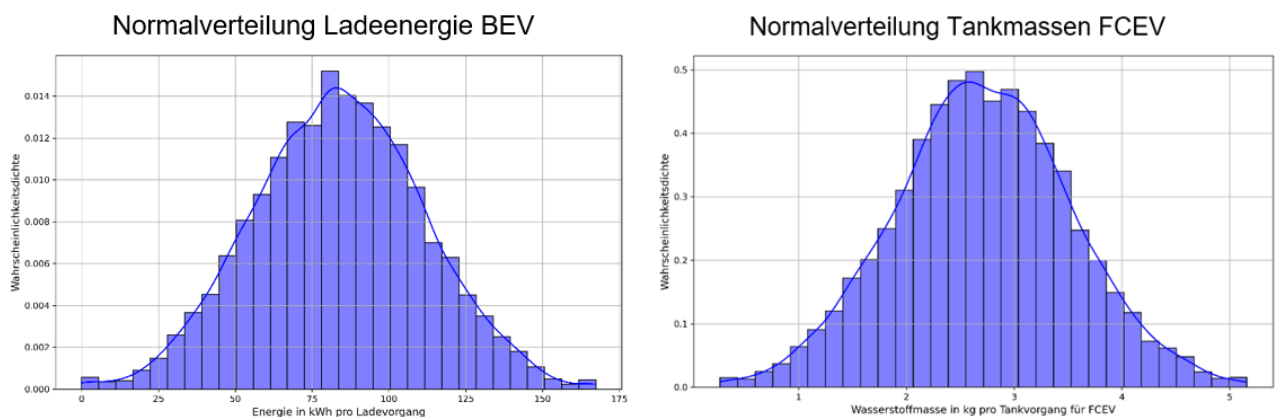
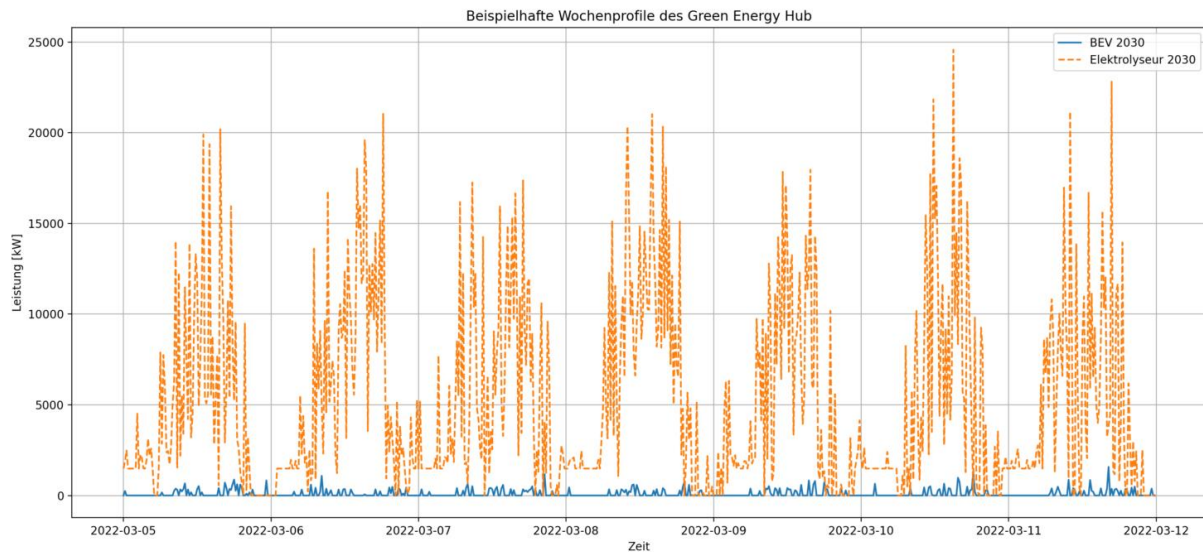


Abbildung: Normalverteilung der Tankmassen (BEV und FCEV)

Die erzeugten Lastgänge dienen als Eingangsdaten für:

- Dimensionierung der H₂-Erzeugungs- und Speichertechnologien
- Simulation der Energieflüsse innerhalb des Green Energy Hub
- Kopplung mit den weiteren Teilprojekten des Innovationsparks



- Beispielhaftes Wochenlastprofil des Green-Energy-Hub-Szenarios 2030
- Die Lastprofile sind Szenario basiert als CSV-Datensätze hinterlegt und werden als zeitaufgelöste Lastgänge in die Energiebilanz integriert

3.4 Industriegebiet

Auswahl Unternehmen:

- TransIS Studie (Wuppertal Institut für Klima, Umwelt, Energie)
- Unternehmen Abfragen aus Masterarbeit von J. M. Wickenhäuser (J. M. Wickenhäuser, Entwicklung eines analytischen Tools zur Bewertung der Netzdienlichkeit von Mikro- und Arealnetzen)

Lastgänge:

1. Anonymisierte Lastgänge von Zenodo (F. Braeuer, Load profile data of 50 industrial plants in Germany for one year, <https://zenodo.org/records/3899018/>)
2. Auswertung der Zenodo Lastgänge
3. Definition jährlicher Lastgang pro Unternehmen
4. Anhand abgefragter Unternehmen aus Masterarbeit skaliert

Definition folgender Unternehmen:

- Entsorgung: 5 1/2 Tage Woche, kein Schichtbetrieb (Anlehnung an Schönackers Umweltdienste (J. M. Wickenhäuser))
- Stahl 1: 5 Tage Woche, 3-Schicht-Betrieb (Anlehnung an TecPro (J. M. Wickenhäuser))
- Logistik und Lagerung: 5 Tage Woche, teilweise Samstag und Sonntag Betrieb, kein Schichtbetrieb (Anlehnung an CTJ Janssen (J. M. Wickenhäuser))
- Schrott- und Metallhandel: 5 Tage Woche, kein Schichtbetrieb (Anlehnung an Willi Jenner (J. M. Wickenhäuser))
- Lagerung: 5 Tage Woche, kein Schichtbetrieb (Anlehnung an Kleine Logistik (J. M. Wickenhäuser))
- Büro: 5 Tage Woche, kein Schichtbetrieb (Anlehnung an J. M. Wickenhäuser)
- Logistik Lebensmittel: 6 1/2 Tage Woche, kein Schichtbetrieb (Anlehnung an CTJ Janssen (J. M. Wickenhäuser))
- Stahl 2: 7 Tage Woche, 2-Schicht-Betrieb (Anlehnung an TecPro (J. M. Wickenhäuser))

Wärme- und Klimatisierungsbedarf für Gebäude:

- nPro Energy (<https://www.npro.energy/>)
- Technologie: reversible Luftwärmepumpen und Geothermie-Sonden

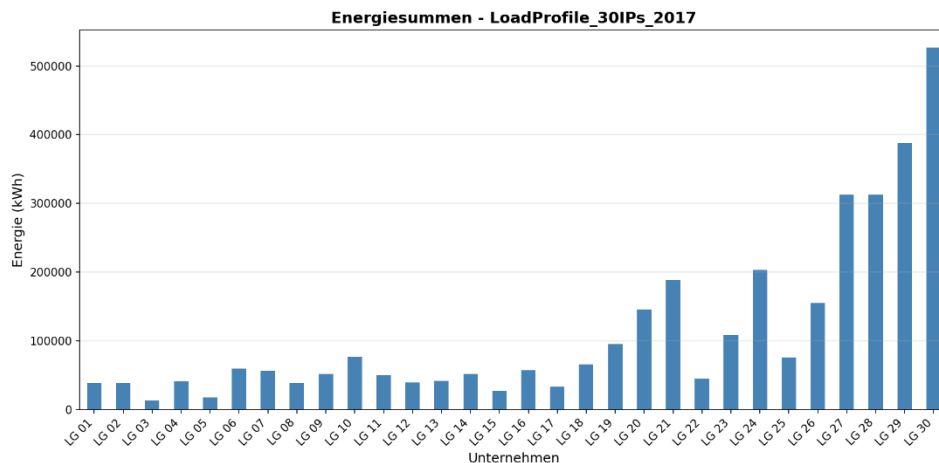
- Gesamtstrombedarf für Industriegebiet ergibt sich aus Lastgängen sowie Wärme- und Klimatisierungsbedarf

3.4.1 Auswahl Lastgänge und Erläuterung Skalierungsfaktor

Berechnung Jahresverbräuche der Zenodo Lastgänge

CSV-Datei: LoadProfile_30IPs_2017.csv (F. Braeuer, Load profile data of 50 industrial plants in Germany for one year, <https://zenodo.org/records/3899018/>)

Lastgang	Jahresverbrauch [MWh]
LG 01	38,47
LG 02	38,18
LG 03	12,94
LG 04	40,86
LG 05	17,57
LG 06	59,25
LG 07	56,43
LG 08	38,54
LG 09	52,11
LG 10	76,47
LG 11	50,44
LG 12	39,38
LG 13	41,96
LG 14	51,44
LG 15	26,88
LG 16	57,49
LG 17	32,85
LG 18	65,47
LG 19	95,20
LG 20	145,69
LG 21	188,54
LG 22	44,58
LG 23	108,19
LG 24	202,87
LG 25	75,68
LG 26	155,14
LG 27	312,88
LG 28	312,90
LG 29	388,04
LG 30	526,55



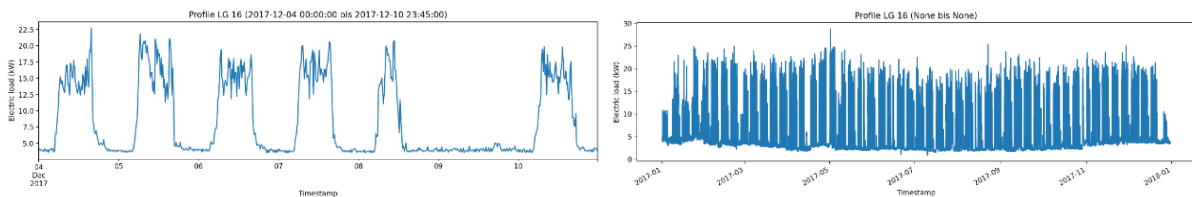
Erläuterung Skalierungsfaktor für Lastgänge

- Lastgänge beziehen sich auf Zenodo „LoadProfile_30IPs_2017“
- Skalierung Fläche anhand von „Datenabfrage Unternehmen“ (Marvin Wickenhäuser) → Fläche der Unternehmen (aus Datenabfrage) zu geplanter Fläche Industriegebiet (aus TransIS Studie, hierbei jedoch Übersichtsrechnung mit 50 000 m² pro Gebäude, entspricht circa 1/5 der bebaubaren Industriefläche (60% der Gesamtfläche))
- Skalierung Verbrauch anhand von Gesamtverbrauch pro Jahr des Lastgangs → Faktor berechnet mit „Verbrauch Skalierungsfaktor Fläche“ geteilt durch „Jahresverbrauch des gewählten Lastgangs“
- Wärme und Kühl Bedarf mithilfe von nPro erstellt (mit Wetterdaten aus Jahr 2022, somit Übereinstimmung mit Wetterdaten für Autobahn und Energielandschaft), Annahme 50 000 m² Fläche pro Gebäude für Wärme und Kühl Bedarf
- Annahme Wärmebedarf: 30 kWh/m²/Jahr
- Annahme Kühlbedarf: 15 kWh/m²/Jahr
- Wärme und Kühl Bedarf:

https://www.bbsr.bund.de/BBSR/DE/veroeffentlichungen/sonderveroeffentlichungen/2023/leitfaden-energiebedarfsausweise-nichtwohnungsbau-2023-dl.pdf?__blob=publicationFile&v=2

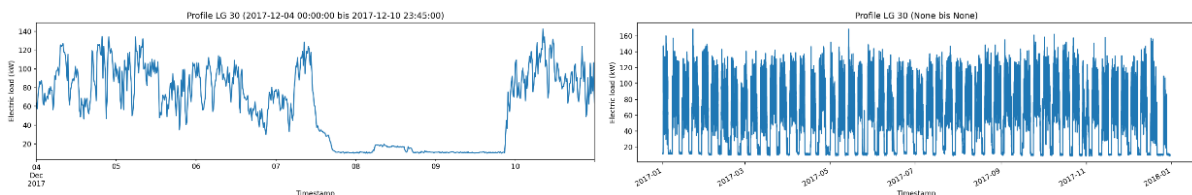
3.4.2 Entsorgung

- Anlehnung an Schönackers Umweltdienste GmbH & Co. KG
- 3 000 000 kWh = 3 000 MWh
- Skalierungsfaktor Fläche: 3 000 MWh * 2,2 = 6 600 MWh
- Skalierungsfaktor Verbrauch: LG 16 * 115
- LG 16
- 5 ½ Tage Woche (Samstag halber Arbeitstag)
- Keine Schicht



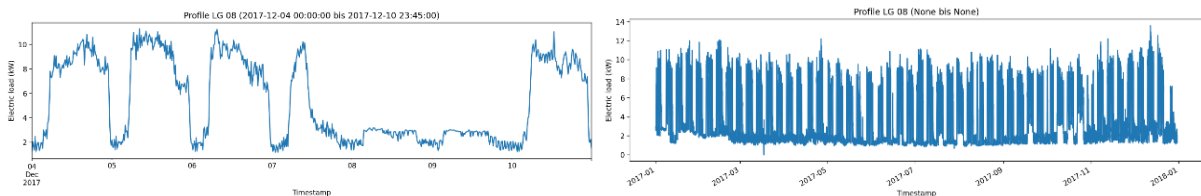
3.4.3 Stahl 1

- Anlehnung an TecPro
- 1 500 000 kWh = 1 500 MWh
- Skalierungsfaktor Fläche: 1 500 MWh * 2,6 = 3 950 MWh
- Skalierungsfaktor Verbrauch: LG 30 * 7,5
- LG 30
- 5 Tage Woche
- 3-Schicht Betrieb



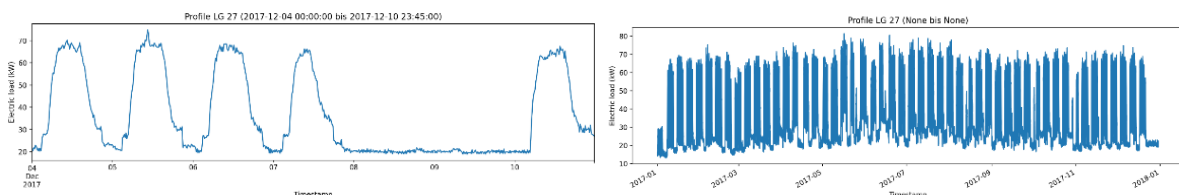
3.4.4 Transportlogistik und Lagerung

- Anlehnung an CTJ Janssen GmbH
- 88 000 kWh = 88 MWh
- Skalierungsfaktor Fläche: $88 \text{ MWh} * 8,3 = 733 \text{ MWh}$
- Skalierungsfaktor Verbrauch: $\text{LG } 8 * 19$
- LG 8
- 5 Tage Woche
- Keine Schicht
- Samstag und Sonntag leichter Verbrauch (Leichte Lagerarbeit, eilige und wichtige Transporte müssen am Wochenende verschickt werden)



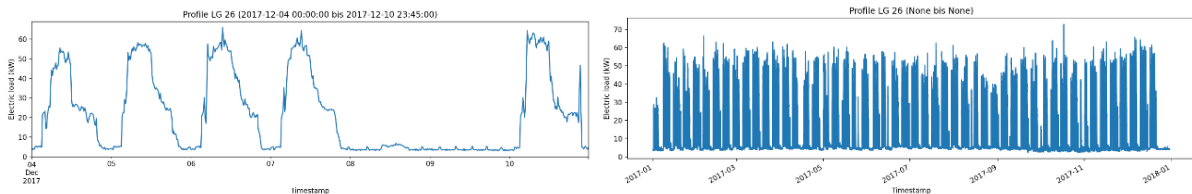
3.4.5 Schrott- und Metallhandel

- Anlehnung an Schrott- und Metallhandel Willi Jenner GmbH
- 20 000 kWh = 20 MWh
- Skalierungsfaktor Fläche: $20 \text{ MWh} * 50 = 1\,000 \text{ MWh}$
- Skalierungsfaktor Verbrauch: $\text{LG } 27 * 3,2$
- LG 27
- 5 Tage Woche
- Keine Schicht



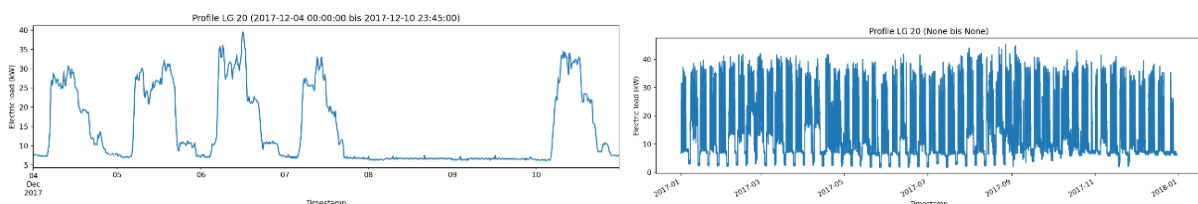
3.4.6 Lagerung

- Anlehnung an Kleine Logistik GmbH
- 2 500 kWh = 2,5 MWh
- Aufgrund von unrealistischer Datenabfrage (Fläche Excel: 36 700 m²), wurde Fläche korrigiert auf 15 000 m²
- Skalierungsfaktor Fläche: 2,5 MWh * 3,3 = 8,3 MWh
- Skalierungsfaktor Verbrauch: LG 26 * 0,054
- LG 26
- 5 Tage Woche
- Keine Schicht
- Nur Lagerung (Lagerhalle), dadurch geringer Verbrauch



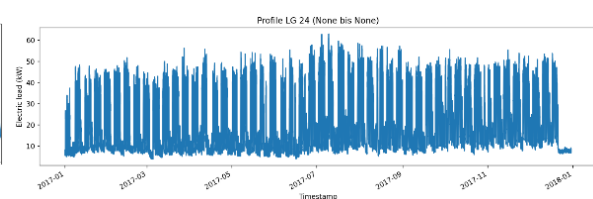
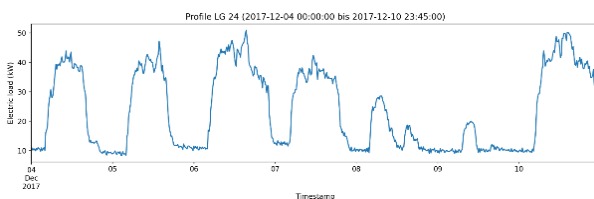
3.4.7 Büro

- Anlehnung an Masterarbeit von Marvin Wickenhäuser
- Annahme: 50 kWh/m² pro Jahr (Annahme von Marvin Wickenhäuser)
- Fläche: 50 000 m²
- 10 MWh (Berechnung mit Excel-Datei von Marvin Wickenhäuser)
- Skalierungsfaktor Verbrauch: LG 20 * 0,069
- LG 20
- 5 Tage Woche
- Keine Schicht



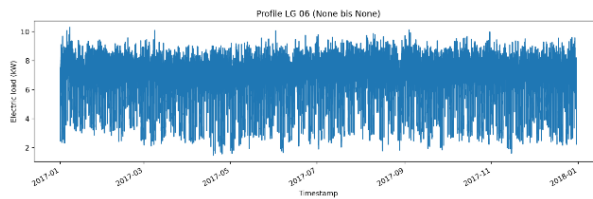
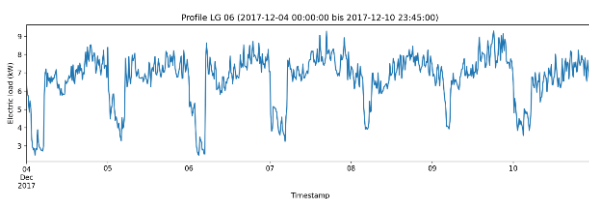
3.4.8 Logistik Lebensmittel

- Anlehnung an CTJ Janssen GmbH
- 88 000 kWh = 88 MWh
- Verbrauch anhand von CTJ Janssen GmbH (Transportlogistik, Lagerung)
- Skalierungsfaktor Fläche: $88 \text{ MWh} * 8,3 = 733 \text{ MWh}$
- Skalierungsfaktor Verbrauch: $\text{LG 24} * 3,6$
- LG 24
- 6 ½ Tage Woche
- Keine Schicht
- Logistik Lebensmittel (Sonntag dürfen Lebensmittel LKW fahren)



3.4.9 Stahl 2

- Anlehnung an TecPro
- 1 500 000 kWh = 1 500 MWh
- Verbrauch anhand von TecPro (Stahl)
- Skalierungsfaktor Fläche: $1\,500 \text{ MWh} * 2,6 = 3\,950 \text{ MWh}$
- Skalierungsfaktor Verbrauch: $\text{LG 6} * 67$
- LG 6
- 7 Tage Woche
- Früh- und Spät Schicht



3.4.10 Weitere Lastgänge

Weitere Lastgänge mit plausiblen Verläufen für Industriegebiet

- LG 13 (5 Tage Woche, keine Schicht, konstanter Verlauf)
- LG 29 (5 Tage Woche, keine Schicht, konstanter Verlauf)
- LG 2 (7 Tage Woche, Früh- und Spät Schicht, konstanter Verlauf)

3.5 Stadtteilentwicklung Jüchen Süd

- Zweckverband LANDFOLGE Garzweiler: Stadt-Teil der Zukunft Jüchen-Süd
(<https://landfolge.de/projekt/stadt-teil-der-zukunft-juechen-sued/>)
- Referenzszenario mit Basispopulation von 1.000 Einwohnern, Grundlage für Skalierung der Ergebnisse
- Spezifischen Wohnfläche von 45 m² pro Kopf
- Differenzierte Siedlungsstruktur: 50% Apartmentblöcken, 30% Mehrfamilienhäusern, 20% Einfamilienhäusern
- Passivhaus-Standard mit Luft-Wärmepumpensystemen
- Wärme-, Kälte- und Strombedarf anhand nPro Energy
(<https://www.npro.energy/>) mit hinterlegten Standardwerte

Annahmen:

- 1.000 Einwohner (wird anschließend im Code hoch/runter skaliert)
 - Ausbaustufe 0: Kein Ausbau
 - Ausbaustufe 1: 1.000 Einwohner
 - Ausbaustufe 2: 2.000 Einwohner
 - Ausbaustufe 3: 3.000 Einwohner
- Ca. 45 m² Wohnfläche pro Kopf
- 50 % Apartmentblock, 30 % Mehrfamilienhaus, 20 % Einfamilienhaus
- Alle Gebäude als Passivhaus inkl. Wärmepumpen
- Lastgänge mit nPro erstellt (nPro Standardwerte für Wärmebedarf, Kältebedarf und Strombedarf)

Gebäudetyp	Bewohner	Nutzfläche (m ²)	Wohneinheiten (WE)
Apartmentblock	500	22.500	350
Mehrfamilienhaus	300	13.500	135
Einfamilienhaus	200	10.000	60
Gesamt	1.000	46.000	545

Nutzungszonen

Nutzungszone	Nutzungstyp	Nutzfläche	Wärme	Kälte	Strom
Apartmentblock	Wohnen	22.500 m ²	720 MWh	405 MWh	495 MWh
Einfamilienhaus	Wohnen	10.000 m ²	400 MWh	180 MWh	220 MWh
Mehrfamilienhaus	Wohnen	13.500 m ²	473 MWh	243 MWh	297 MWh
Summe		46.000 m²	1.592 MWh	828 MWh	1.012 MWh
Max. Leistung			663 kW	1.741 kW	256 kW